



Digital Implementation of Oscillatory Neural Network on FPGA

Prof. Sandip Lashkare

Chinmay Kulkarni, Moksh Jindal

Motivation

- DNNs/CNNs need high memory, power, and compute.
- Von Neumann architectures face bottlenecks for real-time edge tasks.
- ONNs use phase-based computing for efficiency.
- ONNs mimic brain-like sync behavior:
 - Fast parallel computation
 - Robust associative memory
 - Ultra-low power use

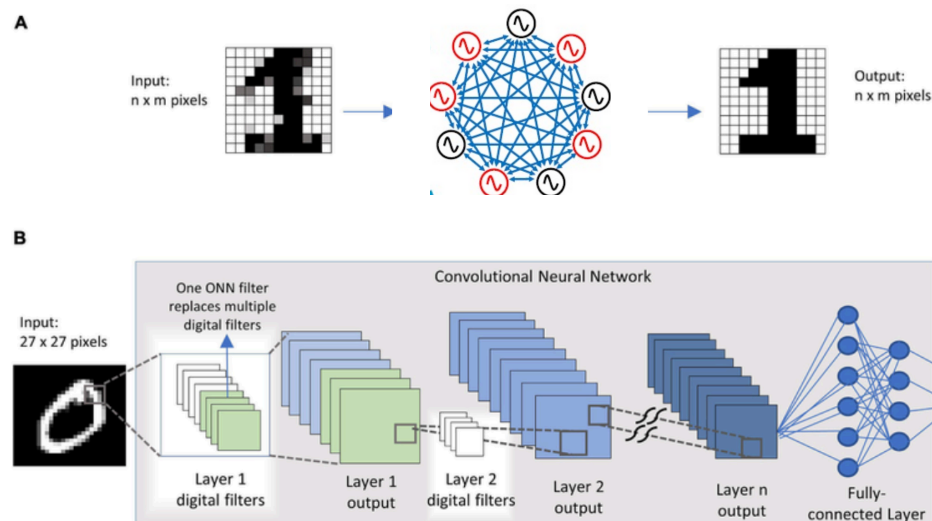


Fig 1. (A) image inference process in an oscillatory neural network; (B) image classification through a convolutional neural network.

Methodology

- We implemented a 15-neuron Oscillatory Neural Network (ONN) on the Nexys 4 DDR FPGA to recognize digits 0, 1, and 2 from 3×5 pixel binary images.
- Digits 0, 1, and 2 were represented as 3×5 binary patterns, where each pixel maps to a neuron in the ONN. Pixel values were encoded as bipolar inputs: +1 for black and -1 for white.

Hebbian Learning The Hebbian learning rule was used to compute the synaptic weights between neurons:

$$w_{ij} = \sum_k \xi_i^k \xi_j^k$$

Fig 2. Hebbian Rule

This produced a 15×15 symmetric weight matrix, which was stored in the FPGA.

Offline training using Python:

Hebbian learning was implemented in Python to compute the synaptic weight matrix from digit patterns. These pre-trained weights were then hardcoded into the FPGA design for inference.

Digital ONN Design

In our digital ONN implementation, we use an arithmetic circuit for each synapse and there is no analog comparator in neurons.

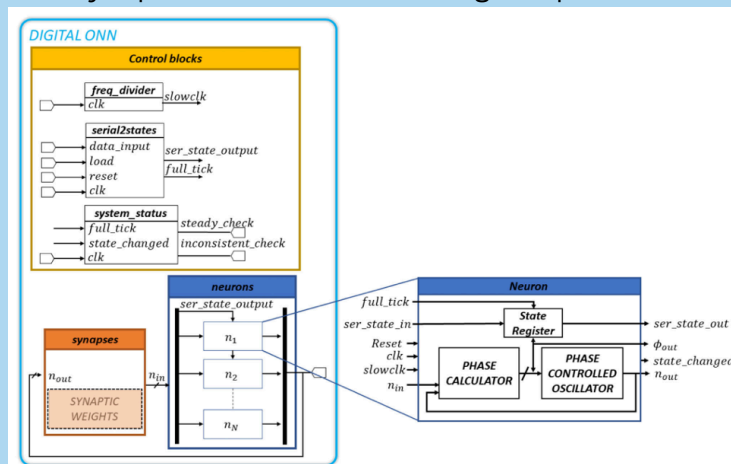


Fig 3. complete digital ONN design

It is a fully parallel design, meaning we implement each neuron inside the FPGA to oscillate in parallel.

Neuron Working:

- Each neuron computes phase difference between its input (ϕ_{in}) and output (ϕ_{out}).
- A Phase Calculator Block handles the update using:
 - Two edge detectors to detect rising edges on n_{in} and n_{out} .
 - A Finite State Machine (FSM) to measure time difference between rising edges.
- The calculated phase difference ($\Delta\phi$) is used to adjust ϕ_{out} .
- The neuron updates its phase to align n_{out} with n_{in} , ensuring synchronization.

$$\phi_{out} = \phi_{out} + / - \Delta\phi$$

The sign (+/-) depends on the first rising edge detected. (-) If n_{out} 's rising edge is detected first, and (+) if n_{in} 's rising edge is detected first.

Synapse Working:

ONN synapses block contains weights and computes each neuron's input oscillations n_{in} . We use an arithmetic logic circuit for our digital ONN design to generate the input signal to the i-th neuron as:

$$n_{in}[i] = \text{sign}(\sum_j w_{ij} - \sum_k w_{ik})$$

where j extends to those neurons with $n_{out}[j]=1$ and k to those with $n_{out}[k]=0$.

Results

- The implemented ONN successfully recognizes digits 0, 1, and 2 from distorted or noisy 3×5 pixel input patterns.
- Despite input corruption (e.g., missing or flipped pixels), the network reliably converges to the correct stored pattern, demonstrating strong auto-associative memory behavior.
- Visual examples below show distorted input patterns and their corresponding correct outputs, highlighting the ONN's robustness in pattern recall.

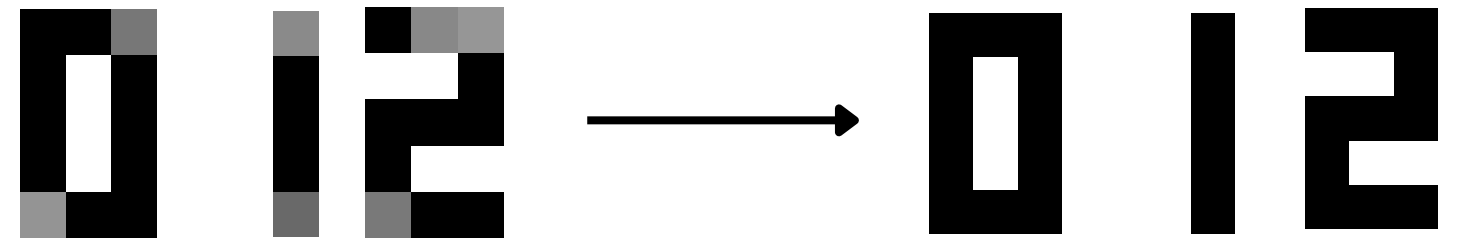


Fig 4 Noisy images after going through ONN

- Results on some tested values are listed below.
- The table contains the number of pixels whose values are deviated from the original image, the intensity of the pixels that are changed, and their position.

ONN	Stored Pattern	Learning Rule	Test Images	Errors	Error rate(%)
5*3	0,1,2	Hebbian	27	5	18.5

Fig 5. Experimental error rate of ONN implemented on FPGA

#Neuron	LUTs(%)	Flip-Flops(%)
15	980(1.54)	654(0.52)

Fig 6. Resource utilization of ONN on Nexys 4 DDR

ONN	Size	Computation time (Clock Cycles)-MAX	Computation time (Clock Cycles)-AVG	Computation time (Clock Cycles)-AVG
hebbian	15	77	50	39

Fig 7. Results on testing noisy images of two

Conclusion

- We built a small brain-inspired network (ONN) on the Nexys 4 DDR FPGA.
- It uses 15 digital neurons to recognize small 3×5 pixel digits (0, 1, and 2).
- The network was trained offline using the Hebbian learning rule.
- Even when parts of the digit were missing or distorted, the ONN could still correctly guess the digit.
- This proves that ONNs can do fast and low-power pattern recognition, making them a good choice for real-time AI on small hardware.

Future Work

- Scale up to 60 neurons (6×10 pixels):
Increase network size to handle higher-resolution inputs for better pattern fidelity.
- Extend to all digits (0–9):
Train the ONN to recognize the full set of decimal digits.
- Integrate camera module input:
Use a live camera feed for real-time digit capture and inference testing.
- Optimize for real-time performance:
Ensure low-latency and power-efficient operation despite increased network complexity.
- Explore resource-efficient scaling:
Investigate trade-offs in LUTs, memory, and timing as the network size grows on the FPGA.

References

- [1] M. Abernot et al., "Digital Implementation of Oscillatory Neural Network for Image Recognition Applications," Frontiers in Neuroscience, vol. 15, Aug. 2021, doi: <https://doi.org/10.3389/fnins.2021.713054>.
- [2] E. Luhulima, M. Abernot, F. Corradi, and A. Todri-Sanial, "Digital Implementation of On-Chip Hebbian Learning for Oscillatory Neural Network," pp. 1–6, Aug. 2023, doi: <https://doi.org/10.1109/islped58423.2023.10244501>.